

CSS Units: Complete Reference

Master em, rem, vh, vw, and More

Contents

1. Introduction	
(a) What are CSS Units?	2
(b) Why Choose Different Units?	2
2. Absolute Units	3
(a) Pixels (px)	3
i. Characteristics	3
ii. Examples	3
(b) Points, Picas, Centimeters	4
i. Other Absolute Units	4
ii. When to Use	4
3. Relative Units	4
(a) Em Unit	4
i. How Em Works	4
ii. Examples	5
(b) Rem Unit	6
i. Root Em vs. Em	6
ii. Examples	6
(c) Percentage (%)	7
i. Percentage Calculations	7
ii. Examples	7
(d) Viewport Units	8
i. vh, vw, vmin, vmax	8
ii. Examples	9
(e) Ex and Ch Units	9
i. Character-Based Units	9
ii. Examples	10
4. Comparison: Absolute vs. Relative	10
(a) When to Use Each Type	10

(b) Practical Recommendations	11
5. Real-World Use Cases	11
(a) Use Case 1: Responsive Font Sizing	11
(b) Use Case 2: Responsive Layouts	12
(c) Use Case 3: Container-Based Design	12
(d) Use Case 4: Hero Sections	13
6. Best Practices	13
(a) Choosing the Right Unit	13
(b) CSS Custom Properties with Units	14
(c) Mobile Considerations	14
7. Advanced Techniques	14
(a) Mixing Units	14
(b) Calc() with Multiple Units	15
(c) Media Query Breakpoints	15
8. Common Mistakes & Solutions	15
(a) Mistake 1: Overusing Fixed Pixels	15
(b) Mistake 2: Nested Em Confusion	16
(c) Mistake 3: Viewport Units on Everything	16
9. Unit Reference Table	16
10. Summary & Conclusion	17

1 Introduction

CSS units define how properties like width, height, font-size, margin, and padding are measured. Understanding when and how to use different units is crucial for responsive, accessible, and maintainable web design.

1.1 What are CSS Units?

CSS units are measurement systems that specify sizes and distances in web design. Every property that requires a size measurement needs a unit.

Two main categories:

- **Absolute Units:** Fixed sizes that don't change (px, pt, cm, mm)
- **Relative Units:** Sizes calculated based on another value (em, rem, %, vh, vw)

1.2 Why Choose Different Units?

Different units serve different purposes:

Need	Best Unit	Reason
Fixed size	px	Consistent across all devices
Responsive text	rem, em	Scales with root/parent font
Screen coverage	vh, vw	Full viewport control
Flexibility	%	Relative to parent element
Typography	em	Perfect for text-based scaling

2 Absolute Units

Absolute units have fixed sizes that never change, regardless of parent elements or viewport size.

2.1 Pixels (px)

Pixels are the most common CSS unit. One pixel equals one dot on the screen.

2.1.1 Characteristics

```
div {  
  width: 300px;  
  height: 200px;  
  font-size: 16px;  
  margin: 10px;  
  padding: 20px;  
}
```

Key points:

- Fixed size - doesn't change based on parent or viewport

- Most intuitive and familiar to beginners
- Predictable and exact
- Can make designs less responsive
- Modern browsers treat 1px as consistent across DPI

2.1.2 Examples

Example 1: Fixed Card Layout

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .card {
      width: 300px;
      height: 400px;
      padding: 20px;
      border: 2px solid #ddd;
      border-radius: 8px;
    }
    .card img {
      width: 260px;
      height: 200px;
      object-fit: cover;
    }
  </style>
</head>
<body>
  <div class="card">
    
    <h3>Card Title</h3>
    <p>Card content here</p>
  </div>
</body>
</html>
```

Example 2: Fixed Navigation Bar

```
<!DOCTYPE html>
<html>
<head>
  <style>
    nav {
      width: 100%;
      height: 60px;
      padding: 0px 20px;
      background: #333;
      display: flex;
      align-items: center;
    }
    nav a {
      margin: 0px 15px;
    }
  </style>
</head>
<body>
  <nav>
    <a href="#home">Home</a>
    <a href="#about">About</a>
    <a href="#contact">Contact</a>
  </nav>
</body>
</html>
```

```
        font-size: 16px;
        color: white;
    }
</style>
</head>
<body>
    <nav>
        <a href="#">Home</a>
        <a href="#">About</a>
        <a href="#">Contact</a>
    </nav>
</body>
</html>
```

2.2 Points, Picas, and Other Absolute Units

Other absolute units exist but are rarely used on the web.

2.2.1 Other Absolute Units

Points (pt):

```
p {
    font-size: 12pt; /* 1 point = 1/72 inch */
}
```

Used in print design and documents. 1pt = 1/72 inch.

Picas (pc):

```
h1 {
    font-size: 2pc; /* 1 pica = 12 points */
}
```

Also used for print. 1pc = 12pt.

Centimeters (cm), Millimeters (mm), Inches (in):

```
@media print {
    body {
        margin: 2cm;
        width: 21cm; /* A4 width */
    }
}
```

These are for print stylesheets only.

2.2.2 When to Use

- pt: Print documents, PDF generation
- pc: Print design layouts
- cm, mm, in: Print stylesheets

- Avoid on-screen use - stick with px or relative units

3 Relative Units

Relative units are calculated based on other values, making them perfect for responsive design.

3.1 Em Unit

The em unit is relative to the font-size of the parent element.

3.1.1 How Em Works

```
/* Parent: font-size 16px */
.parent {
  font-size: 16px;
}

/* Child: 1em = 16px (parent's font-size) */
.child {
  font-size: 1em;      /* 16px */
  font-size: 1.5em;    /* 24px */
  font-size: 0.5em;    /* 8px */
  margin: 1em;         /* 16px */
}
```

Key characteristics:

- 1em = parent element's font-size
- Relative to closest parent with font-size
- Compounds when nested
- Good for spacing related to text
- Can be confusing in deeply nested elements

3.1.2 Examples

Example 1: Text-Based Spacing

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body {
      font-size: 16px;
    }
    h1 {
      font-size: 2em;      /* 32px */
      margin-bottom: 0.5em; /* 8px */
    }
    p {
      font-size: 1em;      /* 16px */
      line-height: 1.5em;  /* 24px */
      margin-bottom: 1em;  /* 16px */
    }
  </style>
</head>
<body>
  <h1>Hello World</h1>
  <p>This is a paragraph</p>
</body>
</html>
```

```
    }
    button {
        font-size: 0.875em;    /* 14px */
        padding: 0.5em 1em;    /* 7px 14px */
    }
</style>
</head>
<body>
    <h1>Heading</h1>
    <p>Paragraph text</p>
    <button>Click Me</button>
</body>
</html>
```

Example 2: Nested Em Problem

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .parent {
            font-size: 16px;
        }
        .child {
            font-size: 1.5em;    /* 24px */
        }
        .grandchild {
            font-size: 1.5em;    /* 36px (nested!) */
        }
    </style>
</head>
<body>
    <div class="parent">
        Parent: 16px
        <div class="child">
            Child: 24px (1.5 × 16)
            <div class="grandchild">
                Grandchild: 36px (1.5 × 24)
            </div>
        </div>
    </div>
</body>
</html>
```

Problem: Em compounds with nesting. Often use rem to avoid this.

3.2 Rem Unit

Rem (Root Em) is always relative to the root element's font-size, solving nesting issues.

3.2.1 Root Em vs. Em


```
/* Root font-size: 16px */
html {
  font-size: 16px;
}

.parent {
  font-size: 20px;
}

.child {
  /* Em: relative to parent (20px) */
  font-size: 1em;      /* 20px */

  /* Rem: always relative to root (16px) */
  font-size: 1rem;     /* 16px */
  margin: 1rem;        /* 16px */
}
```

Key advantages:

- Always references root font-size
- No nesting complications
- Predictable and consistent
- Great for responsive sizing
- Most recommended for modern CSS

3.2.2 Examples

Example 1: Rem-Based Typography

```
<!DOCTYPE html>
<html>
<head>
  <style>
    html {
      font-size: 16px;
    }
    h1 {
      font-size: 2rem;      /* 32px */
      margin-bottom: 1rem;  /* 16px */
    }
    h2 {
      font-size: 1.5rem;    /* 24px */
      margin-bottom: 0.75rem; /* 12px */
    }
    p {
      font-size: 1rem;      /* 16px */
      line-height: 1.5rem;  /* 24px */
      margin-bottom: 1rem;  /* 16px */
    }
  </style>
</head>
</html>
```

```
        button {
            font-size: 1rem;
            padding: 0.5rem 1rem;
            margin: 0.5rem;
        }
    </style>
</head>
<body>
    <h1>Main Title</h1>
    <h2>Subtitle</h2>
    <p>Regular paragraph</p>
    <button>Submit</button>
</body>
</html>
```

Example 2: Responsive Root Font-Size

```
<!DOCTYPE html>
<html>
<head>
    <style>
        html {
            font-size: 16px; /* Base: 16px */
        }

        @media (max-width: 768px) {
            html {
                font-size: 14px; /* Smaller on mobile */
            }
        }

        h1 {
            font-size: 2rem; /* 32px on desktop,
                             28px on mobile */
        }

        p {
            font-size: 1rem; /* 16px on desktop,
                             14px on mobile */
        }
    </style>
</head>
<body>
    <h1>Responsive Heading</h1>
    <p>This text scales with root font-size</p>
</body>
</html>
```

3.3 Percentage (%)

Percentage units are relative to the parent element's corresponding property.

3.3.1 Percentage Calculations

```
.parent {  
    width: 800px;  
    font-size: 16px;  
}  
  
.child {  
    width: 50%;           /* 400px (50% of 800px) */  
    font-size: 80%;       /* 12.8px (80% of 16px) */  
    height: 100%;         /* Same as parent */  
    margin: 10%;          /* 80px (10% of 800px) */  
}
```

Key points:

- Calculated from parent element's value
- Common for width and height
- Can be combined with other units
- Useful for flexible layouts

3.3.2 Examples

Example 1: Responsive Grid

```
<!DOCTYPE html>  
<html>  
<head>  
    <style>  
        .container {  
            width: 1200px;  
            margin: 0 auto;  
        }  
        .column {  
            width: 33.333%;    /* 3 columns */  
            float: left;  
            padding: 20px;  
            box-sizing: border-box;  
        }  
        .column-half {  
            width: 50%;        /* 2 columns */  
        }  
        .column-full {  
            width: 100%;       /* Full width */  
        }  
    </style>  
</head>  
<body>  
    <div class="container">  
        <div class="column">Column 1</div>  
        <div class="column">Column 2</div>  
        <div class="column">Column 3</div>
```

```
    <div class="column-full">Full Width</div>
  </div>
</body>
</html>
```

Example 2: Percentage-Based Sizing

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body {
      font-size: 16px;
    }
    .box {
      width: 80%;          /* 80% of viewport */
      height: 200px;
      margin: 10% auto;   /* Centered */
      padding: 5%;
      background: #f0f0f0;
    }
    .box h2 {
      font-size: 150%;    /* 150% of 16px = 24px */
    }
  </style>
</head>
<body>
  <div class="box">
    <h2>Percentage-Based Content</h2>
  </div>
</body>
</html>
```

3.4 Viewport Units

Viewport units are relative to the browser window size. Perfect for full-screen and responsive designs.

3.4.1 vh, vw, vmin, vmax

1. vw - Viewport Width

```
.element {  
  width: 100vw;    /* 100% of viewport width */  
  width: 50vw;     /* 50% of viewport width */  
  font-size: 5vw;  /* Scales with viewport */  
}
```

1vw = 1% of viewport width.

2. vh - Viewport Height

```
.hero {  
  height: 100vh;   /* Full viewport height */  
  height: 50vh;    /* Half viewport height */  
}
```

1vh = 1% of viewport height.

3. vmin - Viewport Minimum

```
.square {  
  width: 50vmin;   /* 50% of smaller dimension */  
  height: 50vmin; /* Maintains square aspect */  
}
```

1vmin = 1% of the smaller viewport dimension.

4. vmax - Viewport Maximum

```
.element {  
  font-size: 5vmax; /* 5% of larger dimension */  
}
```

1vmax = 1% of the larger viewport dimension.

3.4.2 Examples

Example 1: Full-Screen Hero Section

```
<!DOCTYPE html>  
<html>  
<head>  
  <style>  
    .hero {  
      width: 100vw;  
      height: 100vh;  
      background: linear-gradient(135deg,  
        #667eea 0%,  
        #764ba2 100%);  
      display: flex;
```

```
        align-items: center;
        justify-content: center;
        color: white;
    }
    .hero h1 {
        font-size: 8vw; /* Scales with viewport */
        text-align: center;
    }
    .hero p {
        font-size: 3vw;
    }
</style>
</head>
<body>
    <div class="hero">
        <div>
            <h1>Welcome</h1>
            <p>Responsive Title</p>
        </div>
    </div>
</body>
</html>
```

Example 2: Responsive Aspect Ratio Box

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .video-container {
            width: 80vw;
            height: 45vw; /* 16:9 ratio */
            background: #000;
            margin: 20px auto;
            display: flex;
            align-items: center;
            justify-content: center;
            color: white;
        }
    </style>
</head>
<body>
    <div class="video-container">
        Video or Content Here
    </div>
</body>
</html>
```

Example 3: Fluid Typography

```
<!DOCTYPE html>
<html>
<head>
    <style>
        h1 {
```

```
        font-size: clamp(24px, 5vw, 48px);
        /* Min: 24px, Preferred: 5vw, Max: 48px */
    }
    p {
        font-size: clamp(14px, 2vw, 18px);
    }
</style>
</head>
<body>
    <h1>Fluid Typography</h1>
    <p>Scales smoothly between min and max</p>
</body>
</html>
```

3.5 Ex and Ch Units

Character-based units for precise typography control.

3.5.1 Character-Based Units

1. ex - Height of the letter 'x'

```
.element {
    height: 5ex;    /* 5 times the height of 'x' */
    line-height: 2ex;
}
```

1ex 0.5 times the font-size (varies by font).

2. ch - Width of the character '0'

```
code {
    width: 40ch;    /* 40 characters wide */
    overflow-x: auto;
}
```

1ch = width of the '0' character at current font-size.

3.5.2 Examples

Example 1: Fixed-Width Code Block

```
<!DOCTYPE html>
<html>
<head>
    <style>
        code {
            display: block;
            width: 60ch; /* 60 characters */
            background: #f5f5f5;
            padding: 1rem;
            border-radius: 4px;
            overflow-x: auto;
        }
    </style>
</head>
<body>
    <pre>
        font-family: monospace;
        font-size: 1.2em;
        line-height: 1.2;
    </pre>
</body>
</html>
```

```

        font-family: 'Courier New';
    }
</style>
</head>
<body>
    <code>
        function hello() {
            console.log("Hello World");
        }
    </code>
</body>
</html>

```

Example 2: Typography with Ex Unit

```

<!DOCTYPE html>
<html>
<head>
    <style>
        h1 {
            font-size: 2rem;
            margin-bottom: 1ex; /* Based on x-height */
        }
        p {
            line-height: 1.8ex;
            font-size: 1rem;
        }
    </style>
</head>
<body>
    <h1>Typography</h1>
    <p>Using ex units for spacing</p>
</body>
</html>

```

4 Comparison: Absolute vs. Relative

4.1 When to Use Each Type

Scenario	Use Absolute (px)	Use Relative (rem, em, %)
Font sizes	Limited - only for icons	Preferred
Spacing	Limited	Preferred
Layout	Sometimes	Preferred
Borders	Yes	Acceptable
Media queries	No	Yes
Accessibility	Limited	Excellent

4.2 Practical Recommendations

Font Sizing:

```
/* Good: Responsive */
html { font-size: 16px; }
h1 { font-size: 2rem; }
p { font-size: 1rem; }

/* Avoid: Fixed sizes */
h1 { font-size: 32px; }
p { font-size: 16px; }
```

Spacing:

```
/* Good: Relative to text */
button {
    padding: 0.5rem 1rem;
    margin: 1rem 0;
}

/* Avoid: Fixed pixels */
button {
    padding: 8px 16px;
    margin: 16px 0;
}
```

Layout:

```
/* Good: Flexible */
.container {
    width: 90%;
    max-width: 1200px;
    margin: 0 auto;
}

/* Avoid: Fixed width */
.container {
    width: 1000px;
}
```

5 Real-World Use Cases

5.1 Use Case 1: Responsive Font Sizing

Combine root font-size with rem units for scalable typography.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    html {
      font-size: 16px;
    }
    @media (max-width: 768px) {
      html {
        font-size: 14px;
      }
    }
    @media (max-width: 480px) {
      html {
        font-size: 12px;
      }
    }

    h1 { font-size: 2.5rem; }
    h2 { font-size: 2rem; }
    h3 { font-size: 1.5rem; }
    p { font-size: 1rem; }
    small { font-size: 0.875rem; }
  </style>
</head>
<body>
  <h1>Responsive Typography</h1>
  <p>All text scales with viewport</p>
</body>
</html>
```

5.2 Use Case 2: Responsive Layouts

Use percentages and viewport units for flexible layouts.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .container {
      display: grid;
      grid-template-columns: 1fr 1fr 1fr;
      gap: 2rem;
      padding: 2rem;
    }
    @media (max-width: 768px) {
      .container {
        grid-template-columns: 1fr 1fr;
      }
    }
  </style>
</head>
<body>
  <div class="container">
    <div></div>
    <div></div>
    <div></div>
  </div>
</body>
</html>
```

```
    }  
  }  
  @media (max-width: 480px) {  
    .container {  
      grid-template-columns: 1fr;  
    }  
  }  
</style>  
</head>  
<body>  
  <div class="container">  
    <div class="card">Column 1</div>  
    <div class="card">Column 2</div>  
    <div class="card">Column 3</div>  
  </div>  
</body>  
</html>
```

5.3 Use Case 3: Container-Based Design

Use em units for component-based spacing.

```
<!DOCTYPE html>  
<html>  
<head>  
  <style>  
    .card {  
      font-size: 14px;  
      padding: 1.5em;  
      border: 1px solid #ddd;  
    }  
    .card h3 {  
      font-size: 1.5em;  
      margin: 0 0 0.5em 0;  
    }  
    .card p {  
      margin: 0.5em 0;  
      line-height: 1.5em;  
    }  
    .card button {  
      margin-top: 1em;  
      padding: 0.5em 1em;  
    }  
  </style>  
</head>  
<body>  
  <div class="card">  
    <h3>Card Title</h3>  
    <p>Card content</p>  
    <button>Action</button>  
  </div>  
</body>  
</html>
```

5.4 Use Case 4: Hero Sections

Viewport units for full-screen impact.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .hero {
      width: 100vw;
      height: 100vh;
      background: url('bg.jpg') center/cover;
      display: flex;
      align-items: center;
      justify-content: center;
    }
    .hero-content {
      text-align: center;
      color: white;
    }
    .hero h1 {
      font-size: clamp(2rem, 8vw, 4rem);
      margin-bottom: 1rem;
    }
    .hero p {
      font-size: clamp(1rem, 3vw, 1.5rem);
    }
  </style>
</head>
<body>
  <div class="hero">
    <div class="hero-content">
      <h1>Full Screen Impact</h1>
      <p>Using viewport units</p>
    </div>
  </div>
</body>
</html>
```

6 Best Practices

6.1 Choosing the Right Unit

Decision Tree:

- **Font sizes?** Use rem (primary), em for components
- **Margins/Padding?** Use rem (consistency), em (components)
- **Layout width?** Use %, vw, or max-width
- **Heights?** Use %, vh (be careful with vh)

- **Full screen?** Use 100vh, 100vw
- **Fixed sizes?** Use px (minimal cases)

6.2 CSS Custom Properties with Units

Combine variables with units for maximum flexibility.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --base-size: 16px;
      --base-rem: 1rem;
      --spacing-unit: 1rem;
      --font-size-large: 1.5rem;
      --font-size-small: 0.875rem;
    }

    body {
      font-size: var(--base-rem);
    }

    h1 {
      font-size: var(--font-size-large);
      margin-bottom: var(--spacing-unit);
    }

    @media (max-width: 768px) {
      :root {
        --spacing-unit: 0.75rem;
      }
    }
  </style>
</head>
</html>
```

6.3 Mobile Considerations

Account for smaller screens and touch devices.

```
/* Base: 16px for desktop */
html {
  font-size: 16px;
}

/* Tablet: Reduce slightly */
@media (max-width: 768px) {
  html {
    font-size: 15px;
  }
}
```

```
/* Mobile: Reduce for readability */
@media (max-width: 480px) {
  html {
    font-size: 14px;
  }
}

/* Large screens: Increase for comfort */
@media (min-width: 1920px) {
  html {
    font-size: 18px;
  }
}
```

7 Advanced Techniques

7.1 Mixing Units

Combine different units in one property using `calc()`.

```
.element {
  width: calc(100% - 2rem);
  margin: calc(1rem + 5px);
  font-size: calc(1rem + 0.5vw);
}
```

7.2 Calc() with Multiple Units

Create complex responsive calculations.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .container {
      width: calc(100vw - 20px);
      max-width: 1200px;
    }

    .box {
      height: calc(100vh - 60px - 2rem);
    }

    h1 {
      font-size: calc(1rem + 2vw);
    }

    .column {
      width: calc((100% - 4rem) / 3);
    }
  </style>
```

```
</head>
</html>
```

7.3 Media Query Breakpoints

Define breakpoints using meaningful units.

```
/* Using em for media queries */
@media (max-width: 48em) { /* ~768px */
  /* Tablet styles */
}

@media (max-width: 30em) { /* ~480px */
  /* Mobile styles */
}

@media (min-width: 120em) { /* ~1920px */
  /* Large screen styles */
}
```

8 Common Mistakes & Solutions

8.1 Mistake 1: Overusing Fixed Pixels

Problem: Fixed pixel sizes don't scale with user preferences.

```
/* Bad: Fixed pixels */
h1 {
  font-size: 32px;
}
p {
  font-size: 16px;
  line-height: 24px;
  margin: 20px 0;
}
```

Solution: Use rem for scalable sizing.

```
/* Good: Relative units */
h1 {
  font-size: 2rem;
}
p {
  font-size: 1rem;
  line-height: 1.5rem;
  margin: 1.25rem 0;
}
```

8.2 Mistake 2: Nested Em Confusion

Problem: Em compounds through nesting, causing unpredictable sizes.

```
/* Confusing: Em nesting */
.container { font-size: 2em; }
.child { font-size: 1.5em; }
.grandchild { font-size: 1.5em; } /* Very large! */
```

Solution: Use rem consistently.

```
/* Clear: Rem always references root */
.container { font-size: 2rem; }
.child { font-size: 1.5rem; }
.grandchild { font-size: 1.5rem; } /* Consistent */
```

8.3 Mistake 3: Viewport Units on Everything

Problem: Viewport units scale with viewport, not content.

```
/* Problematic: Everything scales with viewport */
body {
  font-size: 2vw;
  padding: 5vw;
}
h1 {
  font-size: 10vw; /* Too large on mobile */
}
```



```
}
```

Solution: Use `clamp()` or mix units strategically.

```
/* Better: Controlled scaling */
h1 {
  font-size: clamp(1.5rem, 5vw, 3rem);
}
p {
  font-size: clamp(0.875rem, 2vw, 1rem);
}
```

9 Unit Reference Table

Unit	Type	Based On	Use Case
px	Absolute	Pixels	Fixed sizes, borders
em	Relative	Parent font-size	Component spacing
rem	Relative	Root font-size	Typography, spacing
%	Relative	Parent value	Responsive layouts
vw	Relative	Viewport width	Responsive sizing
vh	Relative	Viewport height	Full-screen sections
vmin	Relative	Smaller viewport dim	Flexible sizing
vmax	Relative	Larger viewport dim	Scaling elements
ex	Relative	Letter 'x' height	Precise typography
ch	Relative	Character '0' width	Code blocks
pt	Absolute	1/72 inch	Print documents
cm	Absolute	Centimeter	Print stylesheets

10 Summary & Conclusion

10.1 Key Takeaways

1. **Use rem for typography** - Consistent, scalable, accessible
2. **Use % for layouts** - Flexible, parent-relative sizing
3. **Use viewport units carefully** - Great for full-screen, not for small text
4. **Minimize px usage** - Use only for fixed sizes (borders, icons)
5. **Combine units with calc()** - Create powerful responsive equations
6. **Test on multiple devices** - Ensure units work across screen sizes
7. **Consider accessibility** - Relative units respect user preferences

10.2 Modern CSS Approach

```

:root {
  --font-size-base: 16px;
  --spacing-xs: 0.25rem;
  --spacing-sm: 0.5rem;
  --spacing-md: 1rem;
  --spacing-lg: 1.5rem;
  --spacing-xl: 2rem;
}

body {
  font-size: 1rem;
}

```

```
h1 { font-size: 2.5rem; margin: 1.5rem 0; }
h2 { font-size: 2rem; margin: 1.25rem 0; }
p { font-size: 1rem; margin: 1rem 0; }

@media (max-width: 768px) {
  :root {
    --font-size-base: 14px;
  }
}
```

Understanding CSS units is fundamental to creating responsive, accessible, and maintainable web designs. By choosing the right unit for each situation, you empower users to experience your content comfortably on any device while respecting their accessibility preferences.